Translation strings can also be placed within JSON files, which must be stored in the **`lang`** directory structure. In this approach, each of the application's supported languages will be in a separate JSON file, which will be listed under the **`lang`** directory.

When dealing with a large number of translation strings, this method must be preferred.

```
/lang
  en.json
  es.json
```

Configuring the locale is also trivial in this case. By default, the application's default language is stored in the **`config/app.php`** configuration file under the **`locale`** configuration option.

The default language can be modified for a single HTTP request at runtime with the **`setLocale`** method of the **`App`** facade.

```
use Illuminate\Support\Facades\App;
Route::get('/greeting/{locale}', function (string $locale) {
  if (! in_array($locale, ['en', 'es'])) {
    abort(400);
  }
  App::setLocale($locale);
  // …
});
```

It is also a good practice to configure a fallback language. This should be configured in the **`config/app.php`** configuration file.

The syntax to set a fallback language is as follows:

```
'falback_locale' => '<language>'
```

An example, to set the fallback language as English:

```
'falback_locale' => 'en'
```

To determine the current locale, the **`currentLocale`** and **`isLocale`** methods can be used, as follows:

```
use Illuminate\Support\Facades\App;
$locale = App::currentLocale();
if (App::isLocale('es')) {
  // …
}
```